

OpenMAIC 项目重构计划

版本: v0.2.1 → v1.0.0 (目标)
目标: 将黑盒系统拆解为通俗易懂的技术块组合体，替换所有原创/专有内容，优化多智能体架构，提升可维护性和可扩展性。

一、项目核心文件清单

1.1 入口与配置文件

文件	职责	重要度
app/page.tsx	主页 — 课堂创建/管理 UI	★★★★★
app/layout.tsx	根布局 — ThemeProvider, I18nProvider, AccessCodeGuard	★★★★★
middleware.ts	Edge 中间件 — HMAC 签名 Cookie 访问控制	★★★★★
next.config.ts	Next.js 配置 — standalone 输出, CSP, 代理大小	★★★★
package.json	依赖管理 — 56+ 依赖, monorepo 根	★★★★
.env.example	环境变量模板 — 210 行, 14 个 LLM 提供商	★★★★★

1.2 核心 API 路由

文件	职责	重要度
app/api/chat/route.ts	核心: 多智能体聊天 SSE 端点	★★★★★
app/api/generate-classroom/	异步课堂生成任务	★★★★★
app/api/generate/scene-outlines-stream/	大纲流式生成	★★★★★

文件	职责	重要度
app/api/generate/scene-content/	场景内容生成	★★★★★
app/api/generate/scene-actions/	教师动作生成	★★★★★
app/api/generate/tts/	语音合成	★★★★
app/api/classroom/	课堂 CRUD	★★★★
app/api/pbl/chat/	项目式学习聊天	★★★★

1.3 核心库文件

文件	职责	重要度
lib/orchestration/director-graph.ts	核心: LangGraph 多智能体编排图	★★★★★
lib/orchestration/stateless-generate.ts	核心: 无状态生成入口 + 流式 JSON 解析器	★★★★★
lib/orchestration/prompt-builder.ts	核心: 每个智能体的系统提示词构建	★★★★★
lib/orchestration/director-prompt.ts	导演(调度器)提示词构建与决策解析	★★★★★
lib/orchestration/ai-sdk-adapter.ts	Vercel AI SDK ↔ LangChain 桥接适配器	★★★★★
lib/orchestration/registry/store.ts	智能体注册表 — Zustand store + 默认智能体	★★★★★
lib/orchestration/tool-schemas.ts	动作/工具定义与描述	★★★★★
lib/generation/outline-generator.ts	Stage 1: 需求 → 场景大纲	★★★★★
lib/generation/scene-generator.ts	Stage 2: 大纲 → 完整场景 (最大文件)	★★★★★
lib/generation/scene-builder.ts	无 store 的场景构建器 (用于 SSE 流)	★★★★★
lib/generation/json-repair.ts	鲁棒 JSON 解析 — 多策略修复链	★★★★★
lib/generation/action-parser.ts	LLM JSON 动作解析器	★★★★★

文件	职责	重要度
lib/generation/pipeline-runner.ts	两阶段管线顶层编排	★★★
lib/ai/providers.ts	统一提供商注册表 + 模型工厂	★★★★★
lib/ai/llm.ts	统一 LLM 调用层 (callLLM / streamLLM)	★★★★★
lib/ai/model-metadata.ts	模型能力目录 (thinking/reasoning)	★★★★★
lib/ai/thinking-config.ts	Thinking 配置标准化纯函数	★★★
lib/ai/thinking-context.ts	AsyncLocalStorage thinking 上下文传递	★★★
lib/chat/agent-loop.ts	客户端多智能体循环 — 无状态迭代	★★★★★
lib/store/stage.ts	主状态 store — Stage/Scene/ChatSession	★★★★★
lib/store/settings.ts	设置 store — 提供商配置/模型选择/音频	★★★★★
lib/store/canvas.ts	画布 UI 状态 store	★★★

1.4 类型定义

文件	职责
lib/types/provider.ts	ProviderId, ModelInfo, ThinkingCapability, ThinkingConfig
lib/types/stage.ts	Stage, Scene, SlideContent, QuizContent, InteractiveContent, PBLContent
lib/types/action.ts	Action 联合类型 — 20+ 动作类型
lib/types/chat.ts	ChatSession, StatelessChatRequest, StatelessEvent
lib/types/generation.ts	UserRequirements, SceneOutline, 两阶段类型

1.5 提示词模板 (21 个)

模板目录	用途	需要替换
agent-system	多智能体系统提示词	是 — 核心专有内容
agent-system-wb-teacher	白板教师智能体	是
agent-system-wb-assistant	白板助教智能体	是
agent-system-wb-student	白板学生智能体	是
director	导演调度提示词	是 — 核心专有内容
requirements-to-outlines	Stage 1 大纲生成提示词	是 — 核心专有内容
slide-content / slide-actions	幻灯片内容/动作生成	是
quiz-content / quiz-actions	测验内容/动作生成	是
code-content / diagram-content / game-content / simulation-content / visualization3d-content	Widget 内容生成	是
interactive-outlines / interactive-actions	交互场景	是
pbl-design / pbl-actions	项目式学习	是
widget-teacher-actions	Widget 教师动作	是
web-search-query-rewrite	网络搜索查询重写	是

1.6 环境配置详情

LLM 提供商 (14+2): OpenAI, Anthropic, Google, DeepSeek, Qwen, Kimi, MiniMax, GLM, SiliconFlow, Doubao, OpenRouter, Grok, Tencent, Xiaomi, Ollama(本地), Lemonade(本地)

语音: TTS (OpenAI, Azure, GLM, Qwen, MiniMax, ElevenLabs, Lemonade) / ASR (OpenAI, Qwen, Lemonade)

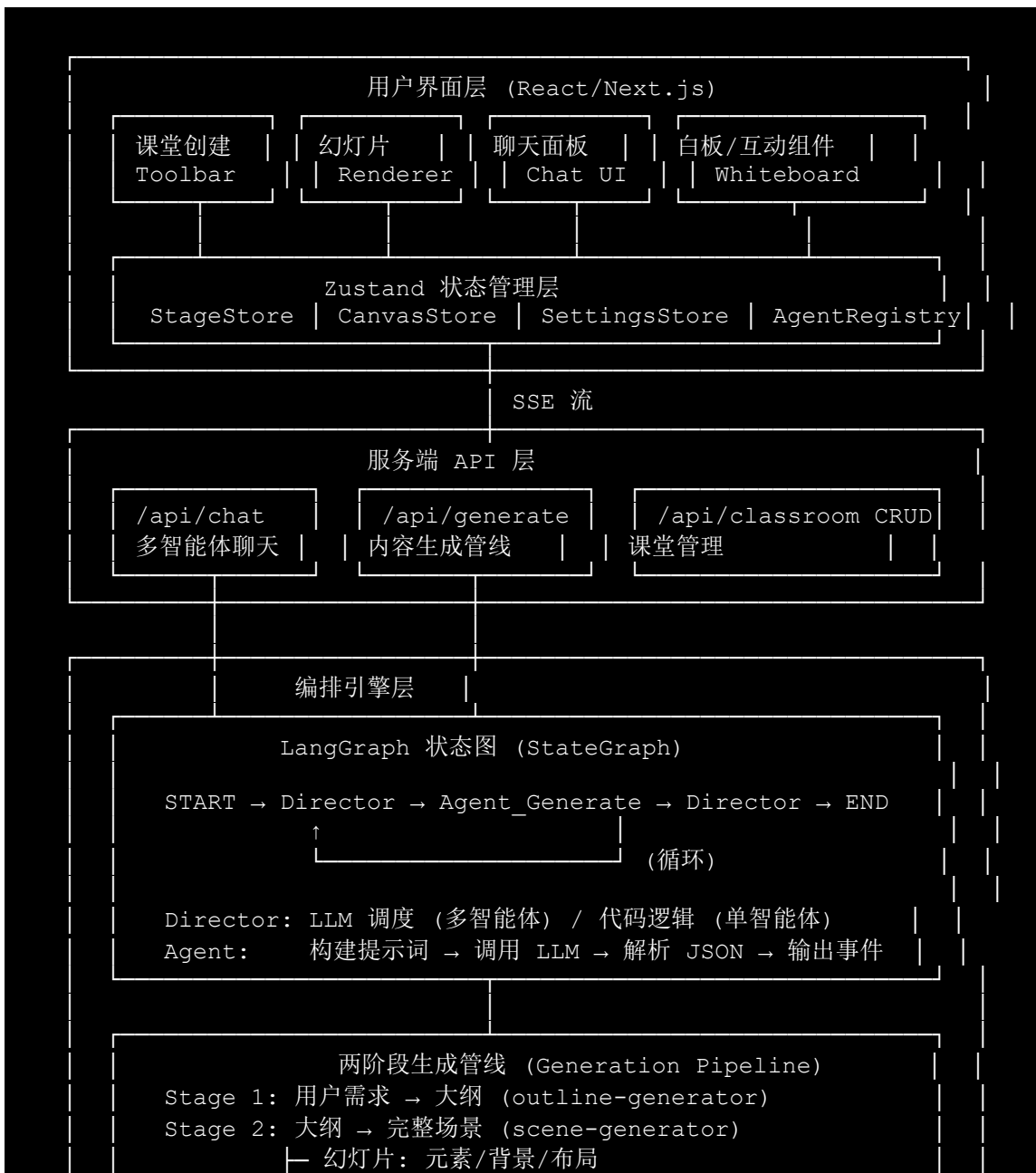
媒体生成: Image (OpenAI, Seedream, Qwen, NanoBanana, MiniMax, Grok, Lemonade) /

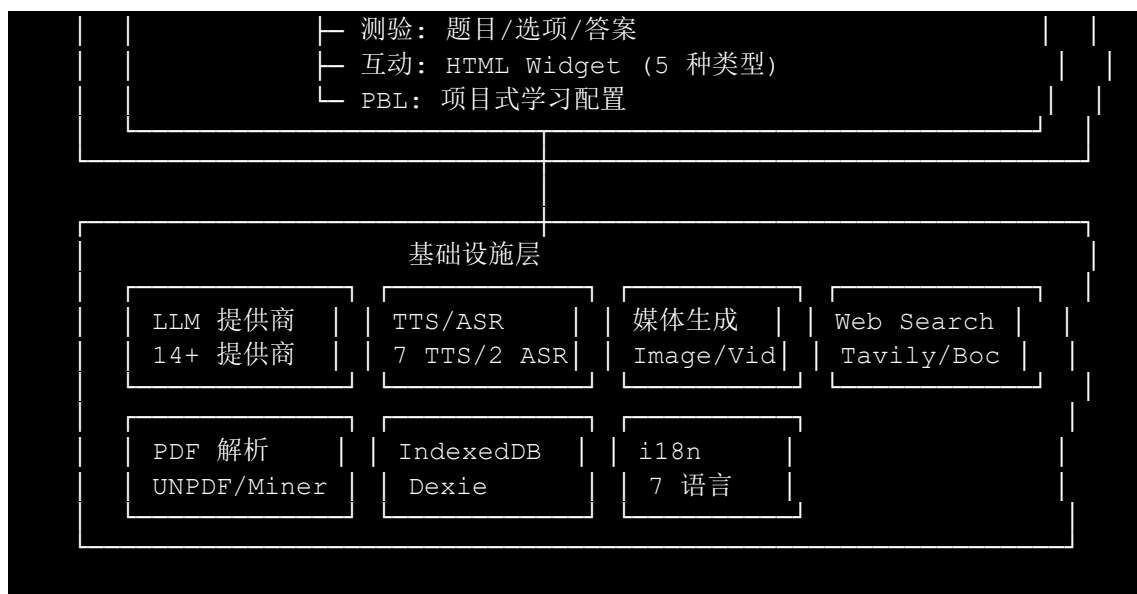
Video (Seedance, Kling, Veo, Sora, MiniMax, Grok, HappyHorse)

其他: PDF (UNPDF, MinerU) / Web Search (Tavily, Bocha) / ACCESS_CODE / 代理设置

二、系统架构拆解 — 黑盒白话化

2.1 整体架构图 (技术块组合体)





2.2 多智能体架构详解

一句话理解: 系统里有 3 类虚拟角色 —— 老师、助教、学生，它们轮流"说话"来模拟一堂课。

核心组件:

1. 导演 (Director) — 决定"下一个该谁说话"

- 输入: 已经说过的话、课程上下文、白板状态
- 输出: `{"next_agent": "teacher-1"}` 或 `END` 或 `USER`
- 单智能体模式: 不用 LLM，纯代码逻辑直接调度
- 多智能体模式: 用 LLM 做路由决策

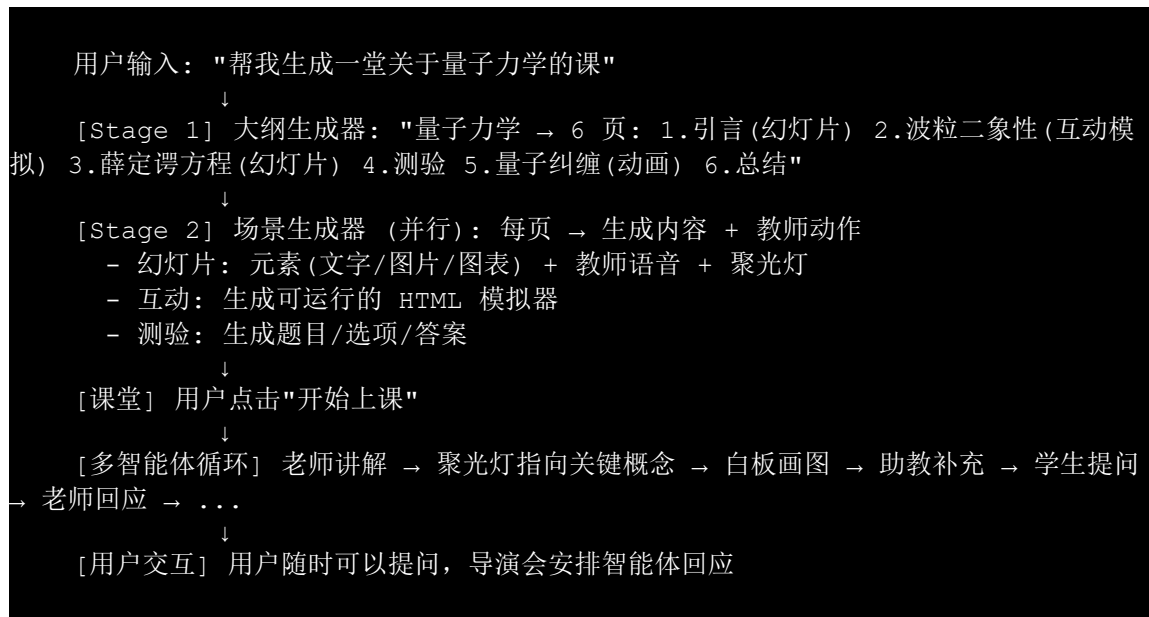
2. 智能体 (Agent) — 实际生成内容的角色

- 收到导演调度后，构建专属提示词 (角色/人设/动作权限/语言约束)
- 调用 LLM 生成结构化 JSON 数组: `[{"type": "text", ...}, {"type": "action", ...}]`
- 文本和动作自由交织: 说一句话 → 聚光灯高亮 → 再说一句话 → 白板画图

3. 循环机制: 导演 → 智能体 → 导演 → 智能体 → ... → 导演决定结束或交给用户

4. 无状态设计: 每次请求携带完整状态 (消息历史 + 课堂数据 + 导演状态)，服务端不存储会话

2.3 数据流白话版



三、需要替换的原创/专有内容标记

3.1 核心专有内容 (必须替换)

内容	位置	替换方案
21 个提示词模板	lib/prompts/templates/* /	全部重写为全新提示词, 基于公开教学法设计
默认智能体人设	lib/orchestration/registry/store.ts 的 6 个默认 agent	替换为通用人设, 名称/描述全部改为英文通用版本
角色行为准则	lib/orchestration/prompt-builder.ts 中 roleGuidelines	重写为通用教学角色定义
导演路由规则	lib/orchestration/director-prompt.ts	重写调度策略为通用对话路由
动作描述系统	lib/orchestration/tool-schemas.ts	重写为通用工具描述
场景生成管线逻辑	lib/generation/scene-generator.ts 中的启发式规	替换 widget 推断逻辑、元素修复规则

内容	位置	替换方案
	则	
教学动作生成	scene-generator.ts 中中文 fallback 语音文本	全部改为英文/多语言
格式化工具	lib/generation/prompt-formatters.ts 中的硬编码文本	替换为可配置模板

3.2 中间层专有内容 (建议替换)

内容	位置	替换方案
白板冲突检测的硬编码画布尺寸	whiteboard-conflicts.ts	提取为常量配置
长度限制 (teacher100, assistant80, student~50)	prompt-builder.ts	提取为可配置参数
课堂时间默认值 (15-20 min)	requirements-to-outlines 模板	提取为用户可配置
Widget 类型选择逻辑	requirements-to-outlines 模板	简化为通用分类
语言推断规则	requirements-to-outlines 模板	简化为显式语言参数

3.3 保留的基础设施代码 (不需要替换)

- Vercel AI SDK 适配层 (ai-sdk-adapter.ts)
- JSON 修复链 (json-repair.ts) — 通用工具
- 流式 JSON 解析器 (stateless-generate.ts 中的 parser)
- 提供商抽象层 (lib/ai/providers.ts) — 通用架构
- Zustand store 架构
- IndexedDB 持久化
- i18n 框架
- shadcn/ui 组件
- Docker/CI 配置

四、重构计划

Phase 1: 基础架构重构

4.1 类型系统标准化

目标: 建立清晰、可扩展的类型层

任务:

- 将 `lib/types/` 下所有类型重构为模块化结构
- 为每个 Action 类型添加 JSDoc 注释说明用途
- 抽取 `ProviderId` 为可扩展的运行时注册制 (而非硬编码联合类型)
- 统一 `Stage / Scene / SceneContent` 的序列化/反序列化接口
- 添加 Zod schema 验证所有 API 输入 (替代手动类型断言)

4.2 提供商层优化

目标: 简化 14+ 提供商的配置, 减少 if/else 分支

任务:

- 将 `getCompatThinkingBodyParams()` 中的 10+ 个 case 分支重构为策略模式
- 提取提供商特定逻辑 (MiniMax URL、Google Proxy) 为独立适配器文件
- 添加提供商能力探测 API (自动检测模型支持的能力)
- 支持热加载提供商配置 (无需重启服务)

新增功能:

- 提供商健康检查仪表板
- 自动故障转移 (当一个提供商不可用时切换到备用)
- 成本追踪 (每个请求的 token 用量和费用估算)

Phase 2: 多智能体重构

4.3 编排引擎重构

目标: 将 LangGraph 编排拆解为可独立测试的模块

任务:

- 将 `director-graph.ts` (548 行) 拆分为:
 - `graph-definition.ts` — 图结构定义 (~100 行)
 - `director-node.ts` — 导演节点逻辑 (~150 行)
 - `agent-node.ts` — 智能体生成节点逻辑 (~200 行)
 - `state-manager.ts` — 状态管理 (~100 行)
- 为导演决策添加单元测试 (当前无测试覆盖编排层)
- 为流式 JSON 解析器添加 `fuzzing` 测试
- 将硬编码的画布尺寸/长度限制等提取为配置

新增功能:

- 智能体插件系统 — 允许第三方扩展智能体类型
- 对话历史压缩 — 长对话自动摘要以减少 token 消耗
- 智能体记忆系统 — 跨轮次记住关键信息

4.4 提示词系统重构

目标: 将 21 个专有模板替换为模块化、可配置的提示词系统

新架构:

```
lib/prompts/
  core/
    agent-base.md          # 基础智能体提示词 (通用框架)
    director-base.md       # 基础导演提示词
    output-format.md       # 输出格式规范
  roles/
    teacher.md             # 教师角色定义
    assistant.md           # 助教角色定义
    student-personas/
      curious.md
      analytical.md
      creative.md
      note-taker.md
  generators/
    outline.md             # 大纲生成
    slide-content.md       # 幻灯片内容
    quiz.md                # 测验
```

```
widget-*.md          # 各类型 Widget
composability.ts     # 提示词组合引擎
```

任务:

- 设计提示词组合引擎 — 将多个模板片段组合成完整提示词
- 支持用户自定义提示词片段 (覆盖默认值)
- 提示词版本管理 (A/B 测试不同提示词的效果)
- 多语言提示词支持 (当前 prompt 是英文硬编码)
- 提示词热更新 (修改后无需重启)

4.5 智能体注册表重构

目标: 将硬编码的 6 个默认智能体替换为可扩展的模板系统

新架构:

```
lib/orchestration/registry/
  types.ts          # AgentConfig, AgentTemplate 接口
  store.ts          # Zustand store (保留)
  templates/        # 智能体模板库
    teacher.ts      # 教师模板
    assistant.ts    # 助教模板
    students/
      curious.ts     # 好奇宝宝
      analyst.ts     # 分析师
      creative.ts    # 创意型
      note-taker.ts  # 笔记员
  factory.ts        # 智能体工厂 — 从模板创建实例
```

任务:

- 将 6 个硬编码智能体改为模板系统
- 支持用户自定义智能体 (通过 UI 创建/编辑)
- LLM 自动生成智能体 (已有的 agent-profiles API 改进为基于模板)
- 智能体组合规则 (哪些角色组合在一起效果最好)

Phase 3: 生成管线重构

4.6 两阶段管线优化

目标: 提升生成速度和质量

任务:

- 将 `scene-generator.ts` (最大文件) 拆分为:
 - `scene-content/` — 各类型内容生成器
 - `slide-generator.ts`
 - `quiz-generator.ts`
 - `interactive-generator.ts`
 - `pbl-generator.ts`
 - `scene-actions/` — 动作生成器
 - `element-fixer.ts` — 元素修复/标准化
 - `media-resolver.ts` — 图片/视频 ID 解析

新增功能:

- 增量生成 — 用户可以边看大纲边修改, 无需等全部完成
- 质量评分 — 每个场景生成后自动评估质量 (完整性、连贯性)
- 生成缓存 — 相似需求复用已有生成结果
- 并行优化 — Stage 2 当前用 `Promise.all`, 改为带并发限制的 `p-map`

4.7 JSON 修复与解析增强

任务:

- 将 `json-repair.ts` 中的修复策略重构为可插拔管道
- 添加结构化输出模式验证 (用 `Zod schema` 校验修复后的 JSON)
- 修复链遥测 — 记录哪些修复策略被触发, 用于优化提示词

Phase 4: 前端架构优化

4.8 状态管理优化

任务:

- 将 `settings.ts` store (已很复杂) 拆分为:
 - `settings/llm.ts` — LLM 提供商配置

- `settings/audio.ts` — 音频配置
- `settings/media.ts` — 媒体生成配置
- `settings/layout.ts` — 布局配置
- `settings/agents.ts` — 智能体配置
- 添加状态变更日志 (调试多智能体交互时的 `state` 变化)
- IndexedDB 迁移系统 — 支持 `schema` 版本升级

新增功能:

- 设置导入/导出 (JSON 文件)
- 设置预设 (教育模式 / 演示模式 / 开发模式)

4.9 组件架构优化

任务:

- 将 `agent-bar.tsx` (41KB) 拆分为多个子组件
- 将 `slide-renderer/Editor/Canvas/hooks/` 中的 ~10 个交互 `hook` 提取为通用 `Canvas` 框架
- 添加组件级 `Error Boundary`
- 虚拟化长列表 (场景列表、聊天历史)

新增功能:

- 暗色/亮色主题完善
- 响应式布局优化 (移动端支持)
- 键盘快捷键系统完善

Phase 5: 基础设施增强

4.10 API 层重构

任务:

- 统一错误处理中间件 (当前各路由分散处理)
- 添加请求速率限制
- API 版本管理 (`/api/v1/...`)
- OpenAPI 文档自动生成

4.11 测试与监控

任务:

- 为编排层添加单元测试 (当前无覆盖)
- 为生成管线添加集成测试
- 添加性能监控 (生成时间、LLM 响应时间、token 消耗)
- 添加错误追踪 (Sentry 或类似方案)

4.12 部署优化

新增功能:

- 环境配置验证器 (启动时检查所有必需的环境变量)
- 健康检查增强 (检查各提供商连接状态)
- Docker 镜像优化 (多阶段构建减小体积)

五、新增功能清单

5.1 教学增强

功能	描述	优先级
自适应难度	根据用户反馈动态调整内容难度	P1
学习路径推荐	基于用户学习历史推荐下一步内容	P2
课堂录制回放	录制智能体对话 + 动作序列，支持回放	P1
实时协作白板	多用户同时编辑白板	P2
课堂导出增强	导出为 PDF/DOCX/SCORM 格式	P1

5.2 智能体增强

功能	描述	优先级
智能体记忆持久化	跨会话记忆用户偏好和学习进度	P1
情感分析	检测用户困惑/满意，调整教学策略	P2
多模态交互	支持语音输入、手势识别	P2
智能体评价系统	用户对智能体表现评分，用于优化	P1

5.3 开发者体验

功能	描述	优先级
CLI 工具	命令行创建/管理课堂	P2
SDK/API	供第三方集成的 REST API	P1
插件市场	社区贡献智能体/模板/Widget	P3
调试面板	实时查看智能体决策过程和 token 消耗	P1

六、优化策略总结

6.1 性能优化

策略	当前问题	优化方案
Token 消耗	每次请求携带完整提示词 (~4000 tokens)	提示词缓存 + 动态提示词压缩
生成速度	Stage 2 串行等待所有场景	带并发限制的并行生成 + 流式输出
首屏加载	无增量加载	大纲生成完成后立即展示，逐页填充内容
内存占用	完整 Stage 存在内存	分页加载 + 场景懒加载

6.2 可靠性优化

策略	当前问题	优化方案
错误恢复	生成失败后无重试	智能重试 + 断点续传 + 降级策略
JSON 解析	多策略修复但无遥测	添加修复遥测 + 结构化验证
状态一致性	无状态设计但状态复杂	添加状态校验 + 版本控制

6.3 可扩展性优化

策略	当前问题	优化方案
提供商扩展	添加新提供商需修改多个文件	插件式提供商注册
智能体扩展	硬编码 6 个智能体	模板系统 + 运行时注册
场景类型扩展	新增场景类型需修改生成管线	策略模式 + 场景处理器注册
Widget 扩展	5 种 Widget 硬编码	Widget 插件系统

6.4 多智能体架构优化

策略	当前实现	优化方案
导演决策	每轮都用 LLM 决策 (token 浪费)	快速路径: 规则引擎预判 + LLM 兜底
对话连贯性	简单截断摘要	分层摘要: 关键信息提取 + 滑动窗口
角色平衡	无保证各角色发言均衡	发言计数器 + 强制轮转兜底
上下文窗口	全量消息传递	智能裁剪: 保留关键对话 + 摘要历史
冲突处理	白板冲突仅告警	主动冲突预防 + 自动布局

七、实施时间表

Week 1-2:	Phase 1	—	基础架构重构	(类型系统 + 提供商层)
Week 3-5:	Phase 2	—	多智能体重构	(编排引擎 + 提示词 + 注册表)
Week 6-7:	Phase 3	—	生成管线重构	(拆分 + 增量生成)
Week 8-9:	Phase 4	—	前端架构优化	(状态管理 + 组件)
Week 10:	Phase 5	—	基础设施增强	(API + 测试 + 部署)
Week 11-12:	集成测试 + 性能调优 + 文档			

八、风险与对策

风险	影响	对策
提示词重构导致生成质量下降	高	A/B 测试 + 保留旧模板作 fallback
LangGraph 升级破坏兼容性	中	锁定版本 + 渐进式升级
多提供商配置迁移复杂	中	自动迁移脚本 + 向后兼容
前端重构引入回归 bug	中	E2E 测试先行 + 组件级快照测试
新智能体系统用户不适应	低	保留经典模式 + 渐进引导